

gocli Developer Reference

System Architecture, Setup Guide, & CLI Operations manual.

What is gocli?

gocli is a high-performance Go-based command line interface designed to streamline developer workflows. It consolidates operations across issue trackers (Jira), source control platforms (GitHub), messaging channels (Microsoft Teams), and CI/CD pipelines, integrating them all directly into the terminal environment. By utilizing local environment settings and secure token store integrations, it eliminates browser context switching and automates repetitive developer processes.

The Developer Loop

The standard workflow supported by **gocli** moves developer tickets from picking to staging in five quick steps:

- **1. Pick Work:** Find active or assigned tickets using `gocli jira list`.
- **2. Start Coding:** Create an aligned branch and move the ticket using `gocli jira branch <KEY>`.
- **3. Create PR:** Construct a GitHub Pull Request using `gocli pr create`.
- **4. AI Review:** Receive immediate automated code critique using `gocli pr review <PR_NUM>`.
- **5. Deploy:** Execute configured deployment recipes on target hosts via `gocli deploy run <NAME>`.

Architecture Overview

Built in Go, **gocli** uses the industry-standard Cobra framework to manage subcommands. Its configurations are maintained under `~/gocli/config.yaml`. Environment-specific secrets are loaded dynamically from `.env` or local session variables to ensure maximum security, preventing tokens from being committed to shared repositories.

System Installation & Configuration

Follow these steps to install the gocli client, configure system access, and authorize the API integrations.

Step 1: Install gocli (PowerShell & Unix One-Liners)

Instead of building from source, you can install the latest compiled binary directly from the terminal. These installers automatically fetch the correct target architecture for your CPU, install the executable, and register the system PATH.

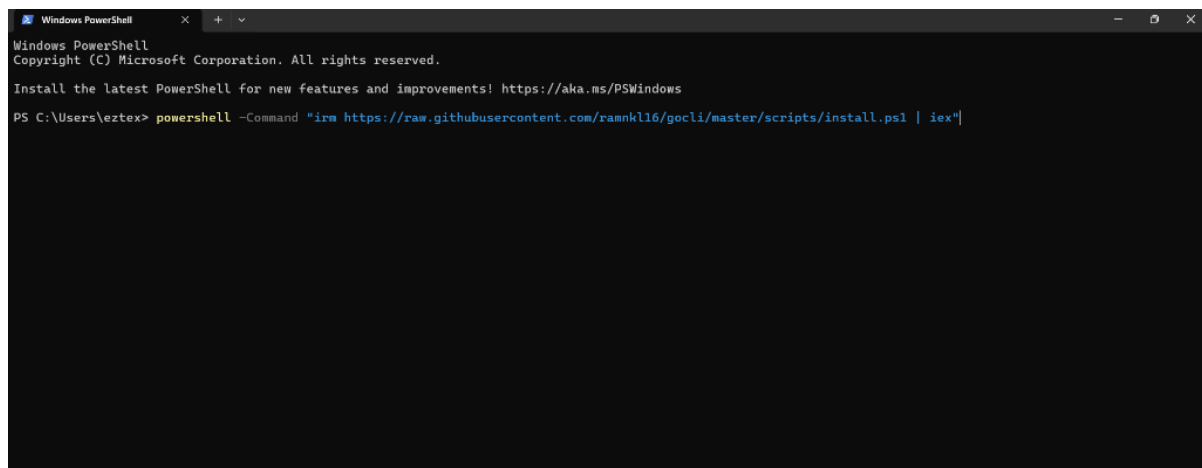
Windows (PowerShell) Automatic Installer:

```
powershell -Command "irm https://raw.githubusercontent.com/ramnkl16/gocli/master/scripts/install.ps1 | iex"
```

macOS & Linux (Bash) Automatic Installer:

```
curl -sSfL https://raw.githubusercontent.com/ramnkl16/gocli/master/scripts/install.sh | bash
```

Windows PowerShell Installation Command:

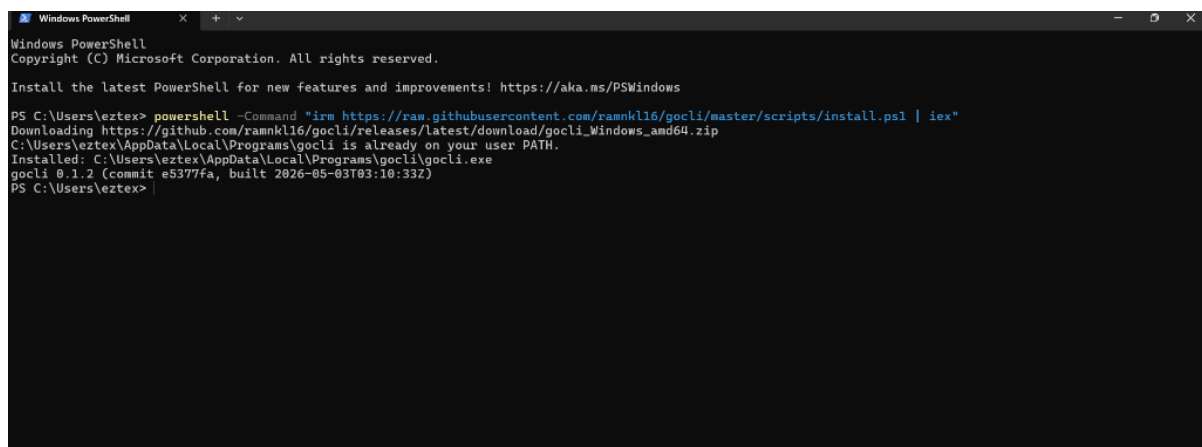


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\eztex> powershell -Command "irm https://raw.githubusercontent.com/ramnkl16/gocli/master/scripts/install.ps1 | iex"
```

Successful Installation Verification:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

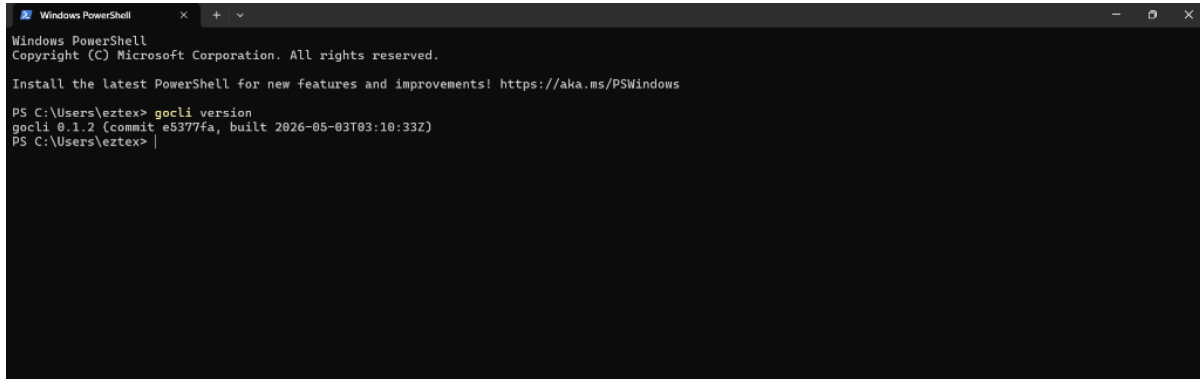
PS C:\Users\eztex> powershell -Command "irm https://raw.githubusercontent.com/ramnkl16/gocli/master/scripts/install.ps1 | iex"
Downloading https://github.com/ramnkl16/gocli/releases/latest/download/gocli_windows_amd64.zip
C:\Users\eztex\AppData\Local\Programs\gocli is already on your user PATH.
Installed: C:\Users\eztex\AppData\Local\Programs\gocli\gocli.exe
gocli 0.1.2 (commit e5377fa, built 2026-05-03T03:10:33Z)
PS C:\Users\eztex>
```

Step 2: Verification

Open a new terminal session to reload your Path environment variables, and verify that the client is working by querying its version:

```
gocli version
```

Successful Version Verification:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ezt\> gocli version
gocli 0.1.2 (commit e5377fa, built 2026-05-03T03:10:33Z)
PS C:\Users\ezt\> |
```

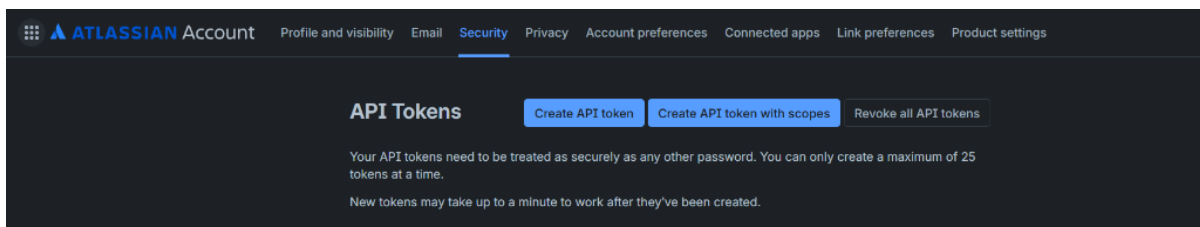
Step 3: Access Token Provisioning

gocli acts as a client bridge requiring authorization credentials for various integrations:

1. Jira API Token:

To generate your Jira API token, follow these steps:

- Navigate to your Atlassian Security Settings page:
<https://id.atlassian.com/manage-profile/security/api-tokens>
- Click the '**Create API token**' button.
- Enter a descriptive label (e.g. gocli) and click **Create**.
- Copy the generated token immediately and store it securely (it will not be displayed again).



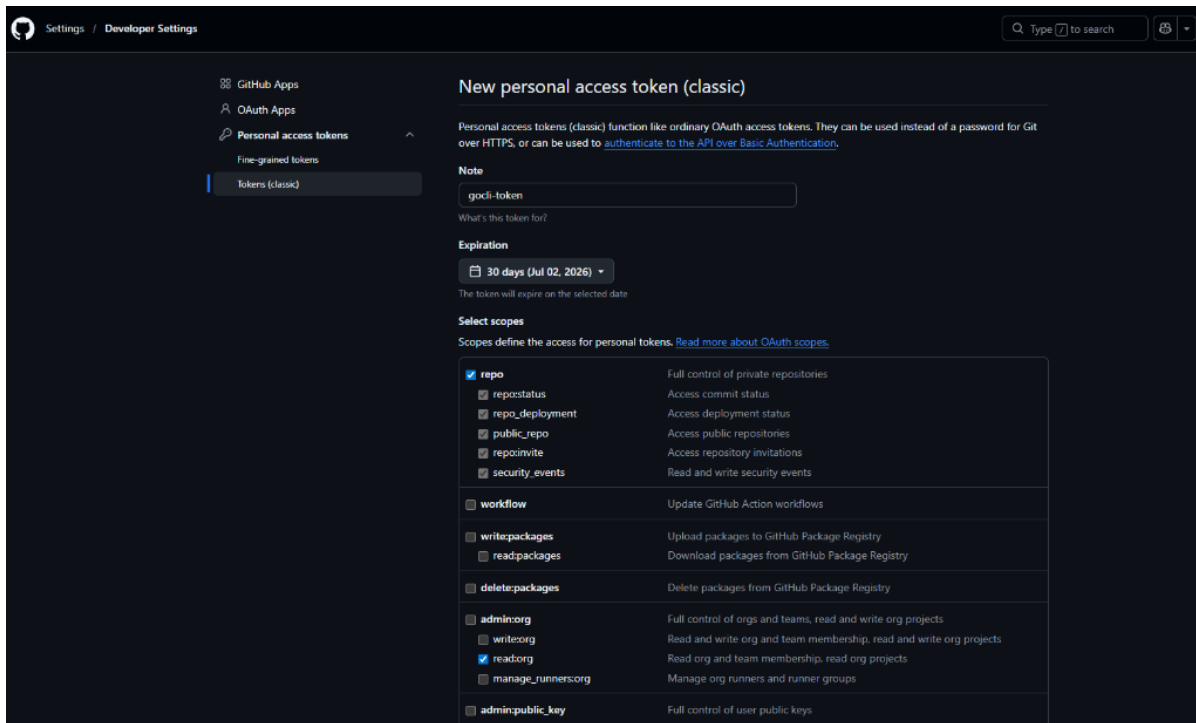
2. GitHub Token:

To generate your GitHub Personal Access Token (classic):

- Navigate to the GitHub Token creation page: <https://github.com/settings/tokens>
- Click '**Generate new token**' and choose '**Generate new token (classic)**'.
- Enter a descriptive note (e.g. gocli-token) and choose an expiration window.
- Select the required scopes: check **repo** (full control of private repositories) and **read:org** (read org and

team membership).

- Click '**Generate token**' at the bottom, then copy the token immediately (it will not be shown again).



3. Teams Webhook:

Established via MS Teams Workflows to broadcast notification cards (see detailed procedure below).

Microsoft Teams Webhook Setup Procedure

Since Microsoft is retiring the classic Incoming Webhook connectors, you must configure webhook integration via the modern **Teams Workflows** system:

- **1. Select Channel:** Open Microsoft Teams, locate your target team channel, click the '...' (**More options**) icon, and select **Workflows**.
- **2. Choose Template:** Search for 'webhook' in the template catalog and select '**Send webhook alerts to a channel**'.
- **3. Confirm Connections:** Verify your user account credentials and click **Next** to authenticate.
- **4. Configure Destination:** Select the specific Team and Channel where alerts should be posted, then click **Add workflow**.
- **5. Copy Endpoint URL:** Once the workflow is successfully created, copy the generated **Webhook URL** link immediately.
- **6. Update Configuration:** Save the URL in your local .env file under `GOCLI_TEAMS_DEVOPS_WEBHOOK`, or add it to your `~/.gocli/config.yaml` under the teams:

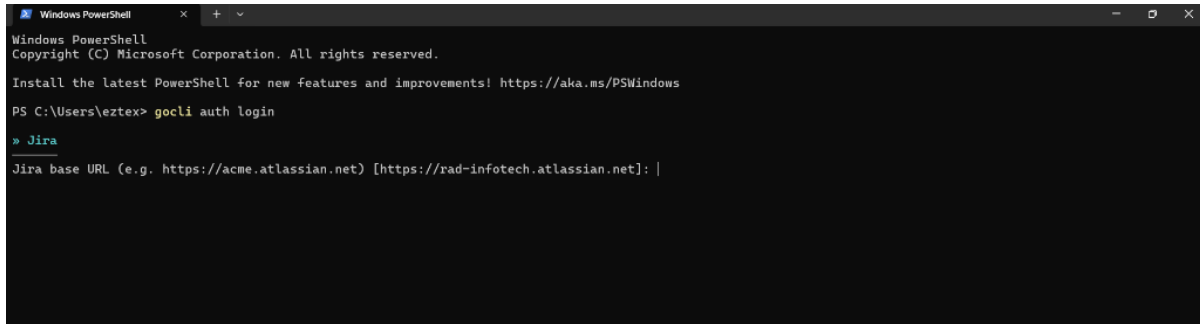
configuration block.

Step 4: Interactive Authorization Login

To initialize and authenticate the integrations for your local workspace, run the secure interactive wizard:

```
gocli auth login
```

Terminal Authentication Wizard Prompt:



Follow the wizard prompts and input the following configuration values:

- **Jira base URL:** Enter your Atlassian Jira domain (e.g. `https://acme.atlassian.net`).
- **Jira email:** Enter your Jira registered account email.
- **Default Jira project:** Enter a default project key (optional, e.g. `ENG`).
- **Jira API token:** Paste the token generated in Step 3.
- **GitHub username:** Enter your personal GitHub username.
- **Default repo:** Specify a default repository path (optional, e.g. `owner/repo`).
- **GitHub token:** Paste the GitHub Personal Access Token generated in Step 3.
- **AI provider / Model:** Select your preferred AI engine for PR reviews (optional, e.g., `github-models / openai/gpt-4o-mini`).

Step 5: Verification of Authentication Status

Confirm that your credentials have been saved and all third-party integrations are successfully configured by querying the CLI status:

```
gocli auth status
```

Terminal Authentication Status Output:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\eztex> gocli auth status
Config: C:\Users\eztex\gocli\config.yaml
SETTING      VALUE      OK
-----
Jira URL      https://rad-infotech.atlassian.net  yes
Jira email    nithishks2004@gmail.com            yes
Jira token    ATA_160                              yes
GitHub user   Nithish0318                         yes
GitHub default repo  no
GitHub token   ghp_Mcn                             yes
AI provider   github-models                       yes
AI model      openai/gpt-4o-mini                 yes
PS C:\Users\eztex>
```

Core CLI Commands Reference

Below is a quick-lookup matrix of supported CLI capabilities across Jira, GitHub, AI review, deployment, and Microsoft Teams notifications.

Area	Command Examples	Function & Scope
Jira	gocli jira list gocli jira view <KEY>	Fetch and format active tickets. View descriptions in clean markdown formats.
Jira	gocli jira branch <KEY> --type bug	Automatic branch creation and transition of the ticket status in a single operation.
GitHub	gocli gh prs gocli gh pr-checkout <N>	Fetch repository pull requests or check out code reviews locally.
GitHub	gocli pr create	Construct upstream pull requests directly from your recent commit history.
AI Review	gocli pr review <N>	Query GitHub Models/OpenAI to run AI sanity checks on active PR diffs.
Deploy	gocli deploy run staging	Trigger secure environment deployment runs specified in deploy files.
Teams	gocli teams notify dev-complete	Alert configured chat rooms of successful developer loop completions.